Chapitre : Le langage Dart pour débutants

Types, fonctions, boucles et classes

Pr. Abdelfattah Abassi

Département Physique Appliquée – Laboratoire LAVETE
Faculté des Sciences et Techniques de Settat
Université Hassan I

Année universitaire 2026–2027

Objectifs et organisation du chapitre

À la fin de ce chapitre, vous saurez

- écrire et exécuter un programme Dart ;
- utiliser les **types** de données et les variables ;
- prendre des décisions (`if`, `switch`) ;
- répéter des actions avec des **boucles** ;
- écrire des **fonctions** réutilisables ;
- modéliser avec des **classes** et des objets.

Prérequis

Aucun. Nous partons de zéro : pas besoin d'avoir déjà programmé.

Méthode

Pour chaque notion : *idée* → *exemple* → *exercice*.

Tapez et testez chaque exemple vous-même !

Plan du chapitre

1. Premiers pas en programmation

2. Variables et types de données

3. Opérateurs et conditions

4. Collections : listes, ensembles et maps

5. Les boucles : répéter des actions

6. Les fonctions

7. Classes et objets

8. Synthèse et mini-projet

Plan du chapitre

1. Premiers pas en programmation

2. Variables et types de données

3. Opérateurs et conditions

4. Collections : listes, ensembles et maps

5. Les boucles : répéter des actions

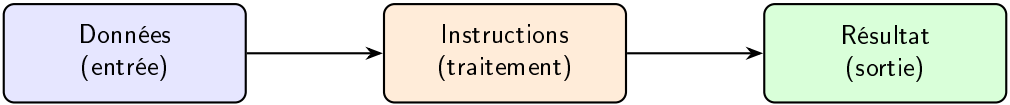
6. Les fonctions

7. Classes et objets

8. Synthèse et mini-projet

Qu'est-ce qu'un programme ?

Un **programme** est une suite d'**instructions** que l'ordinateur exécute, dans l'ordre, pour transformer des **données** en **résultats**.



Analogie : une recette de cuisine

Ingrédients = données • Étapes de la recette = instructions • Plat final = résultat

Algorithme et langage de programmation

Algorithme

Suite **finie** d'étapes **non ambiguës** qui résout un problème.

Langage de programmation

Façon d'écrire un algorithme pour que la machine le comprenne (Dart, Python, Java...).

Compilateur / interpréteur

Outil qui traduit votre code en instructions compréhensibles par le processeur.

Exemple d'algorithme : moyenne de deux notes

1. Lire la note 1
2. Lire la note 2
3. Calculer la somme
4. Diviser la somme par 2
5. Afficher le résultat

Nous écrirons ce programme en Dart dans les prochaines sections.

Pourquoi apprendre Dart ?

- Langage créé par **Google**, moderne et lisible.
- C'est le langage de **Flutter** : un seul code pour Android, iOS, Web et Desktop.
- **Fortement typé** : les erreurs sont détectées avant l'exécution.
- **Null safety** : protège contre les valeurs manquantes.
- Syntaxe proche de Java, C# et JavaScript.
- Utilisable aussi pour le **serveur** et la ligne de commande.

Dans ce cours

Dart → bases du langage
Flutter → interfaces mobiles
IoT → applications connectées

À retenir

Une bonne maîtrise de Dart rend Flutter beaucoup plus facile.

Où écrire et exécuter du Dart ?

Option 1 : DartPad (sans installation)

Site web : `dartpad.dev`
Écrivez le code, cliquez sur **Run**.
Idéal pour débuter.

Option 2 : Dart SDK + VS Code

Installation locale, projet complet, débogueur.
Nécessaire pour lire le clavier (`stdin`).

Commandes dans le terminal :

```
dart --version
dart create mon_projet
cd mon_projet
dart run
```

- `dart create` : crée un projet
- `dart run` : exécute `bin/mon_projet.dart`

Votre premier programme

```dart
void main() {
  // Mon premier programme
  print('Bonjour, monde !');
  print('Bienvenue au cours de Dart');
  print(2 + 3);
}
```

Sortie :

```
Bonjour, monde !
Bienvenue au cours de Dart
5
```

- `main()` : point d'entrée, l'exécution commence ici.
- `void` : la fonction ne renvoie rien.
- `{ ... }` : délimitent un bloc d'instructions.
- `print(...)` : affiche une valeur.
- `;` : termine chaque instruction.
- `// ...` : commentaire ignoré par la machine.

Anatomie d'un programme : commentaires et règles

```dart
// Commentaire sur une ligne

/* Commentaire
   sur plusieurs lignes */

/// Commentaire de documentation

void main() {
  print('Dart');    // commentaire en fin de ligne
  print("Dart");    // guillemets simples ou doubles
}
```

Règles essentielles

- Dart distingue **majuscules** et **minuscules** : `Print` $\neq$ `print`.
- Chaque instruction se termine par ;.
- Indentez votre code : il devient lisible.
- Commentez les parties difficiles.

Les trois familles d'erreurs

Type	Quand ?	Exemple
Erreur de **syntaxe**	Avant l'exécution (le code est mal écrit)	`print('Bonjour')` sans ;
Erreur d'**exécution**	Pendant l'exécution (opération impossible)	`int.parse('abc')` → `FormatException`
Erreur de **logique**	Le programme tourne mais le résultat est faux	`moy = a + b / 2` au lieu de `(a + b) / 2`

Conseil

Lisez toujours le message d'erreur : il indique la **ligne** et la **cause**. Corrigez les erreurs une par une, de haut en bas.

Exercices : premiers pas

1. Dans DartPad, affichez votre nom, votre filière et votre ville sur trois lignes.
2. Affichez le résultat de `(8 + 4) * 3` et de `8 + 4 * 3`. Que remarquez-vous ?
3. Ce code contient 3 erreurs de syntaxe. Trouvez-les :

```dart
void main()
  print('Bonjour);
  Print("Dart");
}
```

Plan du chapitre

1. Premiers pas en programmation

2. Variables et types de données

3. Opérateurs et conditions

4. Collections : listes, ensembles et maps

5. Les boucles : répéter des actions

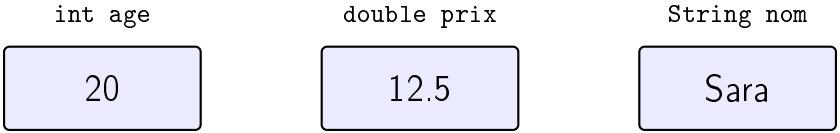
6. Les fonctions

7. Classes et objets

8. Synthèse et mini-projet

La variable : une boîte étiquetée

Une **variable** est un espace mémoire qui porte un **nom**, un **type** et une **valeur**.




```dart
int age = 20;
double prix = 12.5;
String nom = 'Sara';
```

`type nom = valeur;` Le signe = signifie « range la valeur dans la variable ».

Déclarer et nommer une variable

```dart
int age = 20;
double taille = 1.75;
String nom = 'Sara';
bool estInscrit = true;

print('$nom a $age ans');

age = 21;   // modification
print('$nom a $age ans');
```

Sortie :

```
Sara a 20 ans
Sara a 21 ans
```

Règles de nommage

- Lettres, chiffres, _ ; ne commence pas par un chiffre
- Pas d'espace, pas de mot réservé (`class`, `for`...)
- Sensible à la casse

Convention : lowerCamelCase

`noteFinale`, `nombreEtudiants`
(et non `note_finale` ou `N`)

Les types de base de Dart

Type	Rôle	Exemples	Déclaration
`int`	Entiers	42, -7, 0	`int n = 42;`
`double`	Nombres à virgule	3.14, -0.5	`double p = 3.14;`
`num`	`int` ou `double`	5, 5.2	`num x = 5;`
`String`	Texte	`'Dart'`, `"MST"`	`String s = 'Dart';`
`bool`	Vrai / faux	`true`, `false`	`bool ok = true;`

Attention

Le **séparateur décimal** est le point (3.14) et non la virgule. Le texte est toujours entre guillemets.

Nombres : int et double

```dart
int a = 7;
int b = 2;
print(a + b);     // addition
print(a / b);     // division réelle
print(a ~/ b);    // division entière
print(a % b);     // reste
print(10 / 5);    // attention !
double c = 1.5e3;
print(c);
```

Sortie :

```
9
3.5
3
1
2.0
1500.0
```

Piège

L'opérateur / renvoie toujours un `double`, même si le résultat est entier.

Le texte : String

```dart
String a = 'Bonjour';
String b = "Dart";

String c = a + ' ' + b;     // concaténation
String d = '$a $b !';       // interpolation
print(d);

int x = 3, y = 4;
print('Somme = ${x + y}');  // expression

String texte = '''Ligne 1
Ligne 2''';                  // multi-ligne
print('Tab:\tfin\nNouvelle ligne');
```

Sortie :

```
Bonjour Dart !
Somme = 7
Tab:    fin
Nouvelle ligne
```

Interpolation

$variable ou ${expression} insère une valeur dans un texte.

Méthodes utiles sur les String

```dart
String s = 'Flutter';
print(s.length);                  // 7
print(s.toUpperCase());           // FLUTTER
print(s.toLowerCase());           // flutter
print(s[0]);                      // F
print(s.substring(0, 4));         // Flut
print(s.contains('tt'));          // true
print(s.replaceAll('t', 'T'));    // FluTTer
print('a,b,c'.split(','));        // [a, b, c]
print('  mst  '.trim());          // mst
print(s.isEmpty);                 // false
```

Principe

Une **méthode** s'appelle avec un point : `valeur.méthode()`

Remarque

Les index d'un texte **commencent à 0** : `s[0]` est le premier caractère.

Un `String` est immuable : `toUpperCase()` renvoie un nouveau texte.

Les booléens : bool

```dart
bool estMajeur = true;
int age = 17;
bool peutVoter = age >= 18;

print(peutVoter);      // false
print(5 > 3);          // true
print(10 == 10);       // true
print('a' == 'b');     // false
```

Sortie :

```
false
true
true
false
```

- Un `bool` ne peut valoir que `true` ou `false`.
- Il est le résultat d'une **comparaison**.
- Il sert à prendre des décisions (`if`, boucles...).
- En Dart, il n'existe pas de « vrai » implicite : `if (1)` est interdit.

var, final et const

```dart
var ville = 'Settat';  // type inféré : String
ville = 'Casablanca';  // OK
// ville = 42;         // ERREUR de type

final DateTime now = DateTime.now();
// now = DateTime.now(); // ERREUR

const double tva = 0.20;
// tva = 0.25;          // ERREUR

print(ville.runtimeType); // String
```

`var`	type deviné, valeur modifiable
`final`	affectée **une seule fois** (à l'exécution)
`const`	constante connue **dès la compilation**

Bonne pratique

Utilisez `final` par défaut, `var` seulement si la valeur doit changer.

Conversions entre types

```dart
int n = int.parse('42');           // texte -> entier
double d = double.parse('3.14');   // texte -> réel
int? m = int.tryParse('abc');      // null (pas d'erreu

String s = 123.toString();         // '123'
print(3.7.toInt());                // 3
print(5.toDouble());               // 5.0
print(3.6.round());                // 4
print(3.14159.toStringAsFixed(2));// 3.14
```

Règle

Dart ne convertit pas automatiquement un texte en nombre : `'5' + 2` est une erreur.

À utiliser

`parse` : si l'entrée est sûre.
`tryParse` : si l'utilisateur peut se tromper.

Null safety : valeurs absentes

En Dart, une variable **ne peut pas être** `null` sauf si on le demande avec `?`.

```dart
String nom = 'Sara';    // jamais null
String? surnom;         // peut être null

print(surnom);                  // null
print(surnom ?? 'Aucun');       // Aucun
print(surnom?.length);          // null
// print(surnom!.length);       // ERREUR

late String message;    // initialisée plus tard
message = 'Prêt';
print(message);
```

`T?`	type nullable
`??`	valeur par défaut
`?.`	accès sécurisé
`!`	« je suis sûr » (risqué)
`late`	initialisation différée

Lire une saisie au clavier

```dart
import 'dart:io';

void main() {
  stdout.write('Votre prénom : ');
  String? prenom = stdin.readLineSync();

  stdout.write('Votre âge : ');
  int age = int.parse(stdin.readLineSync()!);

  print('Bonjour ${prenom ?? 'inconnu'} !');
  print('Dans un an : ${age + 1} ans');
}
```

- `import 'dart:io';` donne accès à `stdin`/`stdout`.
- `readLineSync()` renvoie un `String?`.
- `stdout.write` affiche sans retour à la ligne.

Attention

DartPad **ne gère pas** `stdin`. Utilisez `dart run` en local.

Exercices : variables et types

1. Déclarez des variables pour : votre nom, votre âge, votre moyenne et un booléen « est étudiant ». Affichez une phrase complète avec l'interpolation.
2. Convertissez le texte `'15.75'` en `double` puis affichez-le arrondi à une décimale.
3. Que valent `17 / 5`, `17 ~/ 5` et `17 % 5` ?
4. Écrivez le programme « moyenne de deux notes » de la section 1, avec des variables `double`.
5. Pourquoi ce code est-il refusé par Dart ?

```dart
int age = 20;
age = 'vingt';
String? s = null;
print(s.length);
```

Plan du chapitre

1. Premiers pas en programmation

2. Variables et types de données

3. Opérateurs et conditions

4. Collections : listes, ensembles et maps

5. Les boucles : répéter des actions

6. Les fonctions

7. Classes et objets

8. Synthèse et mini-projet

Opérateurs arithmétiques

Op.	Rôle	avec 17 et 5
+	addition	22
-	soustraction	12
*	multiplication	85
/	division réelle	3.4
~/	division entière	3
%	reste (modulo)	2

Priorité

*, /, ~/, % avant +, -.

```
print(2 + 3 * 4);      // 14
print((2 + 3) * 4);    // 20
print(17 % 2 == 0);    // false : impair
```

Affectation, incrément et décrément

```dart
int x = 10;
x += 5;     // x = x + 5  -> 15
x -= 3;     // x = x - 3  -> 12
x *= 2;     // x = x * 2  -> 24
x ~/= 5;    // x = x ~/ 5 -> 4

int i = 5;
print(i++);   // affiche 5, puis i = 6
print(++i);   // i = 7, puis affiche 7
i--;          // i = 6
```

Sortie :

```
5
7
```

Raccourcis

`x += n` équivaut à `x = x + n`.

`x++` ajoute 1 à `x`.

Débutants

Écrivez `i++` sur une **ligne seule** pour éviter toute confusion.

Comparaisons et opérateurs logiques

Comparaison → bool

== égal != différent

< > <= >=

Logique

&& ET || OU ! NON

```
int age = 20;
bool ok = age >= 18 && age < 65;
print(ok);   // true
```

A	B	A && B	A || B
true	true	true	true
true	false	false	true
false	true	false	true
false	false	false	false

`!true` vaut `false` et `!false` vaut `true`.

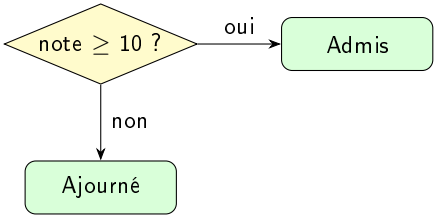
La condition if / else

```dart
int note = 13;

if (note >= 10) {
  print('Admis');
} else {
  print('Ajourné');
}
```

Sortie :

```
Admis
```



La condition entre parenthèses doit être un `bool`.

Plusieurs cas : else if

```dart
double moyenne = 14.5;

if (moyenne >= 16) {
  print('Très bien');
} else if (moyenne >= 14) {
  print('Bien');
} else if (moyenne >= 12) {
  print('Assez bien');
} else if (moyenne >= 10) {
  print('Passable');
} else {
  print('Insuffisant');
}
```

Sortie :

```
Bien
```

Ordre des tests

Les conditions sont testées **de haut en bas** ; dès qu'une est vraie, les autres sont ignorées.

Opérateur ternaire et opérateur ??

```dart
int note = 8;

String resultat = note >= 10 ? 'Admis' : 'Ajourné';   // condition ? siVrai : siFaux
print(resultat);                                      // Ajourné

String? nomSaisi;
String affichage = nomSaisi ?? 'Visiteur';             // valeur par défaut si null
print(affichage);                                     // Visiteur
```

Quand les utiliser ?

Le ternaire convient aux choix **simples** sur une ligne. Pour plusieurs cas, préférez `if` / `else if`.

Choix multiples : `switch`

```dart
int jour = 3;

switch (jour) {
  case 1:
    print('Lundi');
    break;
  case 2:
    print('Mardi');
    break;
  case 3:
    print('Mercredi');
    break;
  default:
    print('Autre jour');
}
```

Version expression (Dart 3) :

```dart
String nom = switch (jour) {
  1 => 'Lundi',
  2 => 'Mardi',
  3 => 'Mercredi',
  _ => 'Autre jour',
};
print(nom);   // Mercredi
```

- `default` / `_` : tous les autres cas.
- Utile pour comparer **une valeur** à plusieurs possibilités.

Exercices : conditions

1. Écrivez un programme qui indique si un entier est **pair** ou **impair**.
2. Donnez la mention d'une moyenne (tableau des mentions : très bien, bien, assez bien, passable, insuffisant).
3. Un nombre *a* est-il compris entre 10 et 20 (inclus) ? Écrivez la condition avec `&&`.
4. Un **tarif** de cinéma : moins de 12 ans $\rightarrow$ 30 DH, entre 12 et 25 ans $\rightarrow$ 40 DH, sinon 60 DH.
5. Avec `switch`, affichez le nom du mois (1 à 12) ; « Mois invalide » sinon.

Plan du chapitre

1. Premiers pas en programmation

2. Variables et types de données

3. Opérateurs et conditions

4. Collections : listes, ensembles et maps

5. Les boucles : répéter des actions

6. Les fonctions

7. Classes et objets

8. Synthèse et mini-projet

Pourquoi des collections ?

Comment stocker les notes de 30 étudiants ? Avec 30 variables ? Non ! On utilise une **collection**.

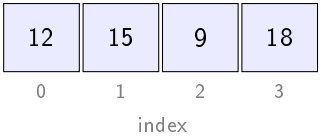
Collection	Idée	Exemple
`List`	suite **ordonnée**, doublons permis	`[12, 15, 9]`
`Set`	valeurs **uniques**	`{'Settat', 'Fès'}`
`Map`	paires **clé** → **valeur**	`{'Maths': 15}`

Type générique

`List<int>` signifie « liste d'entiers » : le type entre < > précise le contenu.

Les listes (List)

```dart
var notes = [12, 15, 9, 18];      // List<int>
print(notes[0]);          // 12
print(notes.length);      // 4
notes.add(14);            // ajoute à la fin
print(notes);             // [12, 15, 9, 18, 14]
notes.remove(9);          // retire la valeur 9
notes[0] = 11;            // modifie l'index 0
print(notes);             // [11, 15, 18, 14]
notes.sort();             // tri croissant
print(notes);             // [11, 14, 15, 18]
print(notes.first);       // 11
print(notes.contains(15)); // true
```



Attention

Le premier élément est à l'index **0**, le dernier à `length - 1`. Accéder à `notes[10]` provoque une `RangeError`.

Ensembles (Set) et dictionnaires (Map)

Set : pas de doublons

```
Set<String> villes = {'Settat', 'Fès', 'Settat'
print(villes);            // {Settat, Fès}
villes.add('Rabat');
print(villes.length);     // 3
print(villes.contains('Fès')); // true
```

Map : clé → valeur

```
Map<String, int> notes = {'Maths': 15, 'Info':
notes['Physique'] = 12;       // ajout
print(notes['Maths']);        // 15
print(notes.keys);            // (Maths, Info,
print(notes.containsKey('Info')); // true
notes.remove('Info');
print(notes.length);          // 2
```

Choisir la bonne collection

Liste → l'ordre compte. Set → pas de doublons. Map → retrouver une valeur à partir d'un **nom** (clé).

Exercices : collections

1. Créez la liste `[14, 8, 17, 11, 6]` : affichez le nombre d'éléments, le premier, le dernier.
2. Ajoutez la note 13, supprimez la note 8, puis triez la liste.
3. Transformez la liste `[1, 2, 2, 3, 3, 3]` en `Set` (utilisez `.toSet()`) : que contient-il ?
4. Créez une `Map` qui associe 3 étudiants à leur âge. Affichez l'âge de l'un d'eux.
5. Que se passe-t-il pour `[1, 2, 3][3]` ? Pourquoi ?

Plan du chapitre

1. Premiers pas en programmation

2. Variables et types de données

3. Opérateurs et conditions

4. Collections : listes, ensembles et maps

5. Les boucles : répéter des actions

6. Les fonctions

7. Classes et objets

8. Synthèse et mini-projet

Pourquoi répéter ?

Sans boucle

Afficher les nombres de 1 à 100 nécessite **100 lignes** `print...` et le code n'est pas modifiable.

Avec une boucle

Quelques lignes suffisent, quel que soit le nombre de répétitions.

Trois boucles en Dart

- `for` : nombre de tours **connu**
- `while` : on répète **tant que** la condition est vraie
- `do-while` : comme `while`, mais au moins une fois

Variante

`for-in` : parcourir une collection.

La boucle for

```dart
for (int i = 1; i <= 5; i++) {
  print('Tour numéro $i');
}
```

Sortie :

```
Tour numéro 1
Tour numéro 2
Tour numéro 3
Tour numéro 4
Tour numéro 5
```

Les 3 parties

1. **Initialisation** : `int i = 1` (une fois)
2. **Condition** : `i <= 5` (avant chaque tour)
3. **Mise à jour** : `i++` (après chaque tour)

Variante

`i += 2` parcourt de 2 en 2, `i--` compte à rebours.

Suivre une boucle à la main : table de trace

```dart
int somme = 0;
for (int i = 1; i <= 4; i++) {
  somme += i;
}
print(somme);   // 10
```

Tour	i	somme avant	somme après
1	1	0	1
2	2	1	3
3	3	3	6
4	4	6	10
–	5	condition fausse : arrêt	

Méthode du débutant

Avant d'exécuter un code, faites sa **trace** sur papier : c'est le meilleur moyen de comprendre une boucle.

La boucle while

```dart
int compteur = 3;

while (compteur > 0) {
  print(compteur);
  compteur--;
}
print('Décollage !');
```

Sortie :

```
3
2
1
Décollage !
```

Principe

La condition est testée **avant** chaque tour. Si elle est fausse dès le départ, le corps ne s'exécute jamais.

Danger : boucle infinie

Si l'on oublie `compteur--`, la condition reste vraie pour toujours !

La boucle do-while

```dart
int x = 10;

do {
  print('x vaut $x');
  x++;
} while (x < 5);
```

Sortie :

```
x vaut 10
```

Principe

Le corps est exécuté **une première fois**, puis la condition est testée.

Usage typique

Demander une saisie à l'utilisateur **au moins une fois**, et recommencer tant qu'elle est invalide.

Parcourir une collection : `for-in`

```dart
List<String> langages = ['Dart', 'Java', 'Python'];

for (String l in langages) {
  print("J'aime $l");
}

Map<String, int> notes = {'Maths': 15, 'Info': 18};
for (var e in notes.entries) {
  print('${e.key} : ${e.value}');
}
```

Sortie :

```
J'aime Dart
J'aime Java
J'aime Python
Maths : 15
Info : 18
```

Avantage

Pas d'index à gérer : on prend chaque élément **un à un**.

break et continue

```dart
for (int i = 1; i <= 10; i++) {
  if (i % 2 == 0) {
    continue;       // passe au tour suivant
  }
  if (i > 7) {
    break;          // quitte la boucle
  }
  print(i);
}
```

Sortie :

```
1
3
5
7
```

continue

Ignore la fin du tour courant et passe au suivant.

break

Arrête **complètement** la boucle.

Boucles imbriquées

```dart
for (int i = 1; i <= 3; i++) {
  String ligne = '';
  for (int j = 1; j <= 4; j++) {
    ligne += '${i * j}  ';
  }
  print(ligne);
}

for (int k = 1; k <= 4; k++) {
  print('*' * k);
}
```

Sortie :

```
1  2  3  4
2  4  6  8
3  6  9  12
*
**
***
****
```

Pour chaque tour de la boucle **externe** (`i`), la boucle **interne** (`j`) fait tous ses tours.

forEach : une écriture compacte

```dart
List<int> nombres = [1, 2, 3];
nombres.forEach((x) => print(x * x));          // 1  4  9

Map<String, int> notes = {'Maths': 15, 'Info': 18};
notes.forEach((matiere, note) => print('$matiere -> $note'));
```

Lecture

`(x) => print(x * x)` est une **fonction anonyme** : « pour chaque `x`, affiche `x * x` ». Nous la détaillerons dans la section sur les fonctions.

Pièges classiques des boucles

1. Boucle infinie

```
int i = 0;
while (i < 5) {
  print(i);
  // oubli de i++
}
```

2. Erreur « d'un cran » (off-by-one)

```
var l = [10, 20, 30];
for (int i = 0; i <= l.length; i++) {
  print(l[i]);   // RangeError
}
```

Correction

Pour un index qui va de 0 à `length - 1`, la condition est `i < l.length` (et non `<=`). Ou mieux : `for (var x in l)`.

Algorithmes classiques : somme, moyenne, maximum

```dart
List<double> notes = [12, 15.5, 9, 18];

double somme = 0;
double max = notes[0];

for (double n in notes) {
  somme += n;
  if (n > max) {
    max = n;
  }
}

print('Moyenne = ${somme / notes.length}');
print('Max = $max');
```

Sortie :

```
Moyenne = 13.625
Max = 18.0
```

Schéma à retenir

Accumulateur : on initialise avant la boucle, on met à jour à chaque tour, on utilise après.

Exercices : boucles

1. Affichez la table de multiplication de 7 (de 7×1 à 7×10).
2. Calculez la somme des nombres pairs de 1 à 100 (résultat attendu : 2550).
3. Calculez $n!$ pour $n = 6$ avec une boucle.
4. Comptez les voyelles du mot `'programmation'` (parcourez le texte avec `for-in`).
5. **FizzBuzz** : pour i de 1 à 30, affichez `Fizz` si i est multiple de 3, `Buzz` si multiple de 5, `FizzBuzz` si multiple des deux, sinon i.

Plan du chapitre

1. Premiers pas en programmation

2. Variables et types de données

3. Opérateurs et conditions

4. Collections : listes, ensembles et maps

5. Les boucles : répéter des actions

6. Les fonctions

7. Classes et objets

8. Synthèse et mini-projet

Pourquoi écrire des fonctions ?

Une **fonction** est un bloc de code **nommé** qu'on peut appeler autant de fois que l'on veut.

Analogie : la machine à café

On donne des ingrédients (**paramètres**), on appuie sur un bouton (**appel**), on récupère un café (**valeur retournée**).

Avantages

- **Réutilisation** : écrire une fois, appeler partout
- **Lisibilité** : un nom clair décrit l'action
- **Maintenance** : une seule correction à faire
- **Tests** plus faciles

Anatomie d'une fonction

```dart
int addition(int a, int b) {
  int resultat = a + b;
  return resultat;
}

void main() {
  int s = addition(3, 4);
  print(s);           // 7
  print(addition(10, 5)); // 15
}
```

`int` (devant)	type de **retour**
`addition`	**nom** de la fonction
`(int a, int b)`	**paramètres**
`{ ... }`	**corps**
`return`	renvoie la valeur

Paramètre ou argument ?

Paramètre : variable dans la définition (`a`, `b`).
Argument : valeur passée à l'appel (3, 4).

Fonction qui ne renvoie rien : `void`

```dart
void saluer(String nom) {
  print('Bonjour $nom !');
}

void main() {
  saluer('Sara');
  saluer('Youssef');
}
```

Sortie :

```
Bonjour Sara !
Bonjour Youssef !
```

`void` ou valeur ?

`void` : la fonction **fait** quelque chose (affiche, modifie...).

Type de retour (`int`, `String`...) : la fonction **calcule** et renvoie un résultat.

Bonne pratique

Une fonction = **une seule tâche**, avec un nom **verbe** : `calculerMoyenne`, `estPair`...

Syntaxe fléchée =>

Quand le corps est **une seule expression**, on peut l'écrire plus court :

```
int carre(int x) {              // forme longue
  return x * x;
}

int carre2(int x) => x * x;     // forme fléchée (équivalente)

bool estPair(int n) => n % 2 == 0;
double moyenne(double a, double b) => (a + b) / 2;

void main() {
  print(carre(5));              // 25
  print(estPair(7));            // false
  print(moyenne(12, 15));       // 13.5
}
```

Paramètres positionnels optionnels

```dart
String saluer(String nom, [String titre = 'M.']) {
  return '$titre $nom';
}

void main() {
  print(saluer('Alami'));          // M. Alami
  print(saluer('Alami', 'Pr.'));   // Pr. Alami
}
```

- Entre [ ] : paramètres **facultatifs**.
- On leur donne une **valeur par défaut** (= 'M.').
- L'ordre des arguments compte.

Paramètres nommés

```dart
double prixTTC({required double prixHT, double tva = 0.25}) {
  return prixHT * (1 + tva);
}

void main() {
  print(prixTTC(prixHT: 100));              // 125.0
  print(prixTTC(tva: 0.5, prixHT: 100));    // 150.0
}
```

- Entre `{ }` : paramètres **nommés**.
- À l'appel : `nom: valeur`, dans **n'importe quel ordre**.
- `required` : obligatoire.
- Sinon : valeur par défaut ou type nullable (`int?`).

Flutter

Les widgets Flutter utilisent massivement les paramètres nommés.

Portée des variables

```dart
int global = 10;              // visible partout

void test() {
  int local = 5;              // visible ici seulement
  print(global + local);      // 15
}

void main() {
  test();
  print(global);              // 10
  // print(local);            // ERREUR : inconnue ici
  for (int i = 0; i < 2; i++) { }
  // print(i);                // ERREUR : i n'existe plus
}
```

Règle

Une variable n'existe que dans le **bloc** { ... } où elle est déclarée.

Conseil

Limitez les variables globales : préférez passer des **paramètres**.

Les fonctions sont des valeurs

```dart
int carre(int x) => x * x;

// une fonction reçoit une autre fonction
int appliquer(int x, int Function(int) f) => f(x);

void main() {
  print(appliquer(5, carre));          // 25
  print(appliquer(5, (n) => n * 2));   // 10  (anonyme)

  var triple = (int n) => n * 3;       // stockée
  print(triple(4));                    // 12
}
```

Fonction anonyme

Fonction **sans nom**, écrite sur place :

`(paramètres) => expression`

Pourquoi ?

Indispensable pour les **callbacks**

Flutter : `onPressed: () { ... }`

Traiter une liste : map, where, reduce

```dart
var notes = [8, 12, 15, 6, 17];

var admis = notes.where((n) => n >= 10).toList();
print(admis);                    // [12, 15, 17]

var bonus = notes.map((n) => n + 1).toList();
print(bonus);                    // [9, 13, 16, 7, 18]

int total = notes.reduce((a, b) => a + b);
print(total);                    // 58

print(notes.every((n) => n >= 10));  // false
print(notes.any((n) => n >= 16));    // true
```

`where`	**filtre**
`map`	**transforme**
`reduce`	**combine**
`every`	tous vrais ?
`any`	au moins un ?

`.toList()` convertit le résultat en liste.

La récursivité

Une fonction **récursive** s'appelle elle-même.

```
int factorielle(int n) {
  if (n <= 1) {
    return 1;                    // cas de base
  }
  return n * factorielle(n - 1);
}

void main() {
  print(factorielle(5));         // 120
}
```

Déroulement :

```
fact(5) = 5 * fact(4)
fact(4) = 4 * fact(3)
fact(3) = 3 * fact(2)
fact(2) = 2 * fact(1)
fact(1) = 1   → 120
```

Obligatoire

Un **cas de base** qui arrête la récursion, sinon : dépassement de pile.

Exercices : fonctions

1. Écrivez `double celsiusVersFahrenheit(double c)` ($F = C \times 1{,}8 + 32$).
2. Écrivez `bool estPremier(int n)` (utilisez une boucle).
3. Écrivez `String mention(double moyenne)` (réutilisez le `if` / `else if`).
4. Écrivez une fonction avec paramètres nommés `creerMessage({required String nom, int age = 18})`.
5. Avec `where` et `map`, à partir de `[3, 8, 12, 5, 20]`, obtenez les carrés des nombres supérieurs à 5.
6. Écrivez `int fibonacci(int n)` de façon récursive.

Plan du chapitre

1. Premiers pas en programmation

2. Variables et types de données

3. Opérateurs et conditions

4. Collections : listes, ensembles et maps

5. Les boucles : répéter des actions

6. Les fonctions

7. Classes et objets

8. Synthèse et mini-projet

Pourquoi des classes ?

Sans classe

```
String nom1 = 'Sara';
int age1 = 20;
String nom2 = 'Youssef';
int age2 = 21;
// et pour 100 étudiants ?
```

Avec une classe

On regroupe les **données** (nom, âge) et les **actions** (se présenter, vieillir) dans un seul modèle : `Etudiant`.

C'est la **programmation orientée objet** (POO).

Classe et objet

Classe

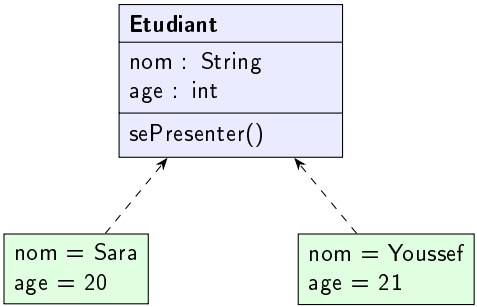
Un **plan** (un moule) : décrit les **attributs** et les **méthodes**.

Objet (instance)

Un exemplaire **concret** créé à partir de la classe, avec ses propres valeurs.

Analogie

Plan d'une maison = classe
Maisons construites = objets



Ma première classe

```dart
class Etudiant {
  String nom = '';
  int age = 0;

  void sePresenter() {
    print('Je suis $nom, $age ans.');
  }
}

void main() {
  Etudiant e1 = Etudiant();    // création d'un objet
  e1.nom = 'Sara';
  e1.age = 20;
  e1.sePresenter();
}
```

Sortie :

```
Je suis Sara, 20 ans.
```

- `class` : définit la classe (nom en **UpperCamelCase**).
- Les **attributs** sont des variables de la classe.
- Les **méthodes** sont des fonctions de la classe.
- `objet.attribut` : accès avec le point.

Le constructeur

Un **constructeur** initialise l'objet dès sa création.

```dart
class Etudiant {
  String nom;
  int age;

  Etudiant(this.nom, this.age);   // raccourci

  void sePresenter() =>
      print('Je suis $nom, $age ans.');
}

void main() {
  var e = Etudiant('Youssef', 21);
  e.sePresenter();
}
```

Forme équivalente (longue)

```dart
Etudiant(String nom, int age) {
  this.nom = nom;
  this.age = age;
}
```

`this` désigne **l'objet courant**. `this.nom` est l'attribut, `nom` seul est le paramètre.

Constructeurs : paramètres nommés et constructeurs nommés

```dart
class Etudiant {
  final String nom;
  final int age;
  String filiere;

  // paramètres nommés avec valeur par défaut
  Etudiant({required this.nom, required this.age, this.filiere = 'MST'});

  // constructeur nommé
  Etudiant.invite() : nom = 'Invité', age = 0, filiere = 'Aucune';
}

void main() {
  var e1 = Etudiant(nom: 'Imane', age: 22);
  var e2 = Etudiant.invite();
  print('${e1.nom} - ${e1.filiere}');   // Imane - MST
  print('${e2.nom} - ${e2.filiere}');   // Invité - Aucune
}
```

`final` : l'attribut ne peut plus être modifié après la création.

Encapsulation : attributs privés

```dart
class CompteBancaire {
  final String titulaire;
  double _solde = 0;            // privé (préfixe _)

  CompteBancaire(this.titulaire);

  double get solde => _solde;   // lecture seule
  void deposer(double montant) {
    if (montant > 0) _solde += montant;
  }

  bool retirer(double montant) {
    if (montant > 0 && montant <= _solde) {
      _solde -= montant;
      return true;
    }
    return false;
  }
}
```

```dart
var c = CompteBancaire('Sara');
c.deposer(500);
print(c.retirer(200)); // true
print(c.solde);        // 300.0
print(c.retirer(1000));// false
```

Idée

On **protège** les données : on ne les modifie que par des méthodes qui vérifient les règles.

En Dart, _ rend le membre privé à la **bibliothèque** (le fichier), pas seulement à la classe.

Getters et setters

```dart
class Etudiant {
  String nom;
  int _age;

  Etudiant(this.nom, this._age);

  int get age => _age;

  set age(int valeur) {
    if (valeur >= 0 && valeur < 120) {
      _age = valeur;
    }
  }

  // getter calculé
  bool get estMajeur => _age >= 18;
}
```

```dart
var e = Etudiant('Sara', 17);
print(e.estMajeur);  // false
e.age = 18;          // setter
print(e.estMajeur);  // true
e.age = -5;          // refusé
print(e.age);        // 18
```

Remarque

On utilise un getter/setter **comme un attribut** (sans parenthèses).

Membres statiques et toString

```
class Etudiant {
  static int compteur = 0;        // partagé par tous
  final String nom;
  final int age;

  Etudiant(this.nom, this.age) {
    compteur++;
  }

  @override
  String toString() => 'Etudiant($nom, $age ans)';
}

void main() {
  var a = Etudiant('Sara', 20);
  var b = Etudiant('Youssef', 21);
  print(a);                       // Etudiant(Sara, 20 ans)
  print(Etudiant.compteur);       // 2
}
```

static

Appartient à **la classe**, pas à un objet : on l'appelle avec le **nom de la classe**.

toString()

Définit comment l'objet s'affiche.
Sans elle : `Instance of 'Etudiant'`.

L'héritage

```
class Personne {
  String nom;
  Personne(this.nom);
  void sePresenter() {
    print("Je m'appelle $nom.");
  }
}

class Etudiant extends Personne {
  String filiere;
  Etudiant(String nom, this.filiere) : super(nom);

  @override
  void sePresenter() {
    super.sePresenter();            // appel au parent
    print('Je suis en $filiere.');
  }
}
```

```
var e = Etudiant('Sara', 'MST');
e.sePresenter();
```

Sortie :

```
Je m'appelle Sara.
Je suis en MST.
```

- `extends` : **hérite** des attributs et méthodes.
- `super` : accès au parent.
- `@override` : redéfinit une méthode.

Classes abstraites et polymorphisme

```
abstract class Forme {
  double aire();                    // sans corps
}

class Cercle extends Forme {
  final double r;
  Cercle(this.r);
  @override
  double aire() => 3.14159 * r * r;
}

class Rectangle extends Forme {
  final double l, h;
  Rectangle(this.l, this.h);
  @override
  double aire() => l * h;
}
```

```
List<Forme> formes = [
  Cercle(2),
  Rectangle(3, 4),
];
for (Forme f in formes) {
  print('Aire = ${f.aire()}');
}
```

Sortie :

```
Aire = 12.56636
Aire = 12.0
```

Polymorphisme : un même appel (`aire()`) donne un comportement différent selon l'objet.

Les énumérations (enum)

```dart
enum Niveau { licence, master, doctorat }

void main() {
  Niveau n = Niveau.master;
  print(n);           // Niveau.master
  print(n.name);      // master

  String texte = switch (n) {
    Niveau.licence  => 'Bac+3',
    Niveau.master   => 'Bac+5',
    Niveau.doctorat => 'Bac+8',
  };
  print(texte);       // Bac+5
}
```

Intérêt

Un **ensemble fini** de valeurs nommées : moins d'erreurs qu'avec des textes libres comme `'mster'`.

Exemples

Statut d'une commande, jours de la semaine, type de capteur IoT...

Exercices : classes

1. Créez une classe `Livre` (titre, auteur, nbPages) avec constructeur et méthode `afficher()`.
2. Ajoutez un getter `estLong` qui vaut `true` si `nbPages > 300`.
3. Créez `Rectangle` avec `aire()` et `perimetre()` ; testez avec deux objets.
4. Créez `Animal` (nom, méthode `crier()`) puis `Chien` et `Chat` qui redéfinissent `crier()`. Stockez-les dans une `List<Animal>`.
5. Ajoutez à `CompteBancaire` une méthode `virer(CompteBancaire autre, double m)`.

Plan du chapitre

1. Premiers pas en programmation

2. Variables et types de données

3. Opérateurs et conditions

4. Collections : listes, ensembles et maps

5. Les boucles : répéter des actions

6. Les fonctions

7. Classes et objets

8. Synthèse et mini-projet

Mini-projet : gestion des notes d'une classe

Cahier des charges

- Classe `Etudiant` : `nom` et liste de `notes`.
- `ajouterNote(double)` : refuse les notes hors de [0 ; 20].
- Getter `moyenne` et getter `mention`.
- `toString()` : « Sara : 16.25 (Très bien) ».
- Dans `main` : créer 3 étudiants, leur ajouter des notes, **trier** par moyenne décroissante, afficher le classement, la moyenne de la classe et le meilleur étudiant.

Notions utilisées

Types, conditions, boucles, collections, fonctions anonymes, classe, getters, `toString()`.

Mini-projet : la classe `Etudiant` (1/2)

```dart
class Etudiant {
  final String nom;
  final List<double> notes = [];
  Etudiant(this.nom);
  void ajouterNote(double note) {
    if (note >= 0 && note <= 20) {
      notes.add(note);
    } else {
      print('Note invalide : $note');
    }
  }
  double get moyenne {
    if (notes.isEmpty) return 0;
    double somme = 0;
    for (final n in notes) {
      somme += n;
    }
    return somme / notes.length;
  }
```

- `notes` : liste de `double` initialisée vide.
- `ajouterNote` : **condition** `&&` pour valider.
- `moyenne` : **boucle** avec accumulateur, cas de la liste vide traité.

La classe se poursuit sur la diapositive suivante.

Mini-projet : la classe `Etudiant` (2/2)

```dart
class Etudiant {
  // ... attributs, ajouterNote, moyenne

  String get mention {
    final m = moyenne;
    if (m >= 16) return 'Très bien';
    if (m >= 14) return 'Bien';
    if (m >= 12) return 'Assez bien';
    if (m >= 10) return 'Passable';
    return 'Insuffisant';
  }

  @override
  String toString() =>
      '$nom : ${moyenne.toStringAsFixed(2)} ($mention)';
}
```

- `mention` : getter qui réutilise `moyenne`.
- Les `return` successifs remplacent le `else if`.
- `toString()` : **interpolation** et arrondi à 2 décimales.

Mini-projet : le programme principal (1/2)

```dart
void main() {
  final classe = [
    Etudiant('Sara'),
    Etudiant('Youssef'),
    Etudiant('Imane'),
  ];

  classe[0].ajouterNote(15);
  classe[0].ajouterNote(17.5);
  classe[1].ajouterNote(9);
  classe[1].ajouterNote(12);
  classe[2].ajouterNote(11);
  classe[2].ajouterNote(14);
  classe[2].ajouterNote(13);
  classe[2].ajouterNote(25);   // refusée

  // ... suite : classement et statistiques
}
```

Sortie : (à ce stade)

```
Note invalide : 25.0
```

Remarque

La note 25 est refusée par `ajouterNote` : l'objet `Etudiant` **protège** ses données.

Mini-projet : le programme principal (2/2)

```
void main() {
  // ... création des étudiants et des notes

  classe.sort((a, b) => b.moyenne.compareTo(a.moyenne));

  print('--- Classement ---');
  for (final e in classe) {
    print(e);
  }

  double total = 0;
  for (final e in classe) {
    total += e.moyenne;
  }
  print('Moyenne de la classe : '
      '${(total / classe.length).toStringAsFixed(2)}');
  print('Major : ${classe.first.nom}');
}
```

Sortie : (programme complet)

```
Note invalide : 25.0
--- Classement ---
Sara : 16.25 (Très bien)
Imane : 12.67 (Assez bien)
Youssef : 10.50 (Passable)
Moyenne de la classe : 13.14
Major : Sara
```

Moyenne de la classe
$= (16{,}25 + 12{,}67 + 10{,}5)/3 \approx 13{,}14.$

Erreurs fréquentes du débutant

Erreur	Correction
Oubli du ;	Terminer chaque instruction par ;
= au lieu de == dans une condition	= affecte, == compare
7 / 2 attendu entier	Utiliser 7 ~/ 2
Index hors limites (RangeError)	Index de 0 à `length - 1`
Boucle infinie	Vérifier que la condition peut devenir fausse
Utiliser une variable `String?` comme `String`	Tester `null`, ou utiliser `??` / `?.`
Oublier `return` dans une fonction typée	Toute branche doit renvoyer une valeur
Oublier `super(...)` ou `this.`	Initialiser tous les attributs du constructeur

Récapitulatif du chapitre

Types et variables

`int`, `double`, `String`, `bool` ; `var`, `final`, `const` ; `null safety` (`?`, `??`, `?.`) ; `List`, `Set`, `Map`.

Conditions

`if` / `else if` / `else`, `switch`, ternaire `? :`

Boucles

`for`, `while`, `do-while`, `for-in`, `break`, `continue`, `forEach`.

Fonctions

Paramètres positionnels / nommés / optionnels, `=>`, fonctions anonymes, `map` / `where` / `reduce`, récursivité.

Classes

Attributs, méthodes, constructeurs (positionnels, nommés), encapsulation (`_`, getters/setters), `static`, `toString`, héritage, classes abstraites, polymorphisme, `enum`.

Pour aller plus loin

Prochaines notions

- Programmation **asynchrone** : `Future`, `async` / `await`, `Stream`
- Gestion des **exceptions** : `try` / `catch`
- Consommer une **API REST** (JSON, `http`)
- Interfaces Flutter : widgets et état

Ressources

- `dart.dev/language` : documentation officielle
- `dart.dev/codelabs` : tutoriels guidés
- `dartpad.dev` : éditeur en ligne
- `api.dart.dev` : référence de l'API

Conseil final

On apprend à programmer en **programmant** : refaites chaque exemple sans regarder la solution !